Otimização Combinatória

Busca Tabu

Matheus Raffael Simon

2025



Introdução

Proposta em 1986 por Fred Glover (GLOVER, 1986)

Surgiu como uma técnica para guiar uma heurística de busca local tradicional na exploração do espaço de soluções além da otimalidade local. Usa, para isso, estruturas de memória (tabelas)

Indicado para problemas de otimização em que a quantidade de soluções é um conjunto discreto mas que cresce exponencialmente

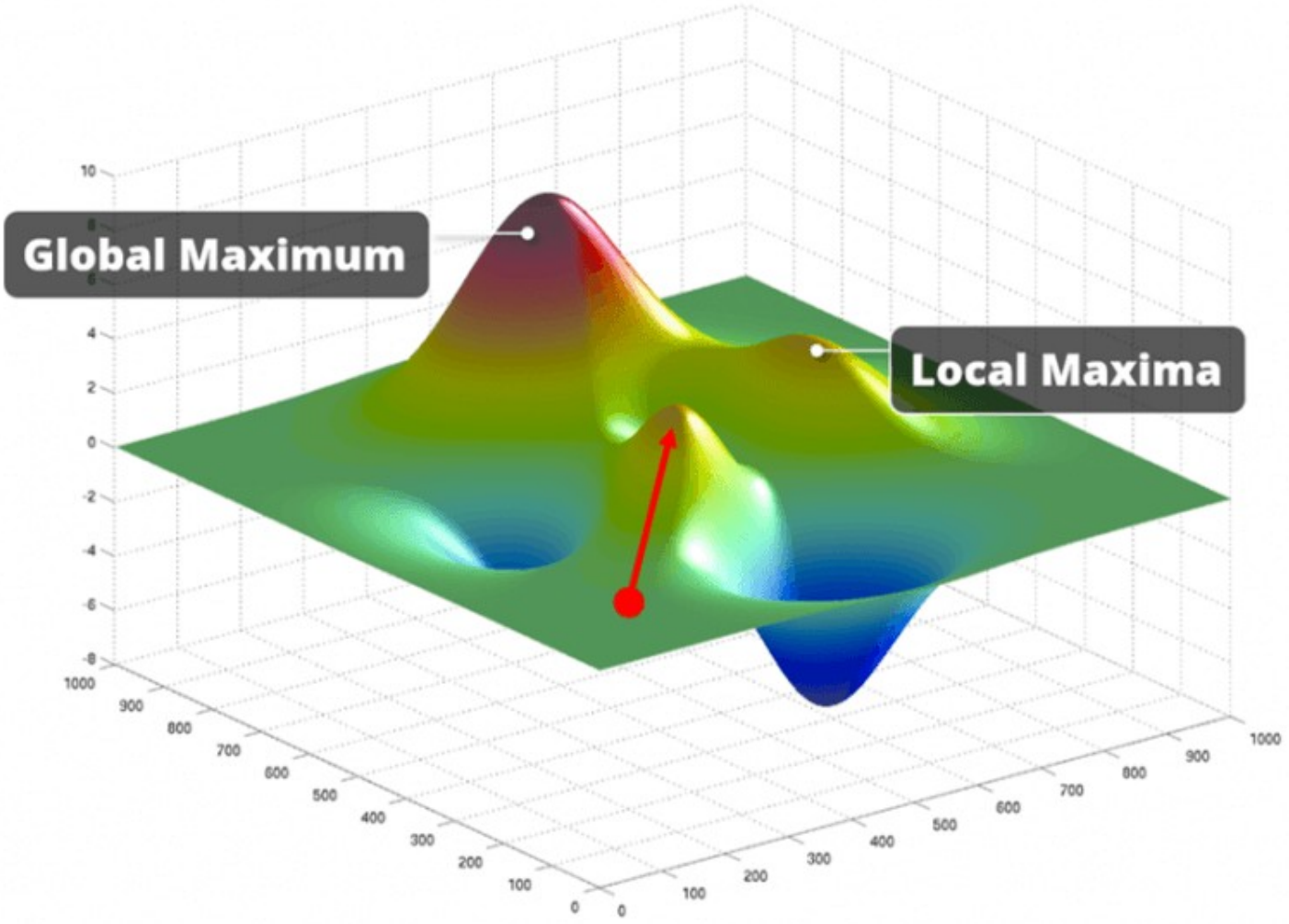


Introdução

Parte de uma solução inicial e se move no espaço de soluções de uma possibilidade para outra que esteja em sua vizinhança

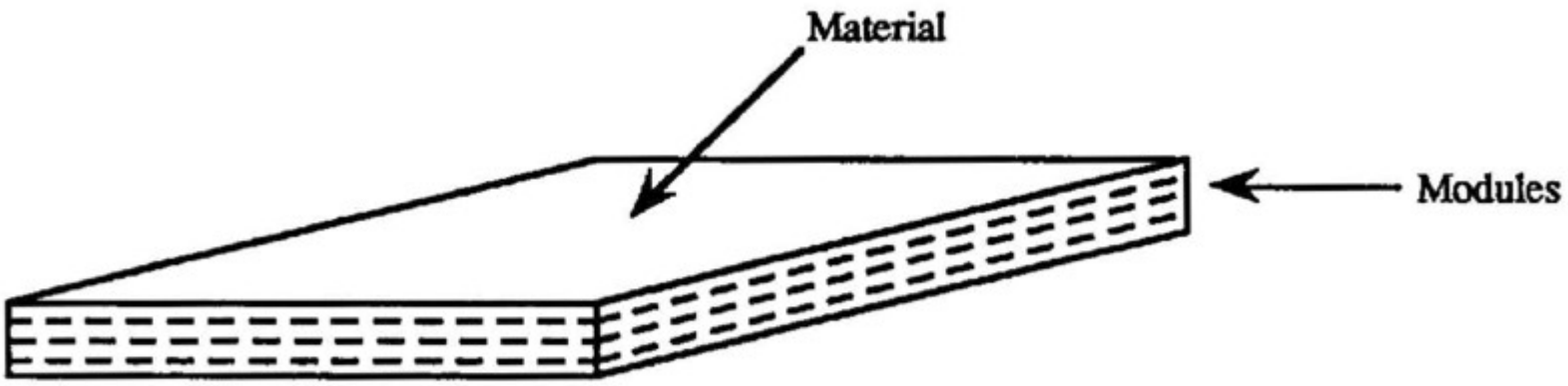
Tenta evitar o recálculo de soluções de forma a otimizar o tempo de execução. Para isso, emprega um mecanismo de armazenamento de soluções já conhecidas e alternativas proibidas, dai o nome Tabu

Introdução



Fonte: Rapidminer (2022)

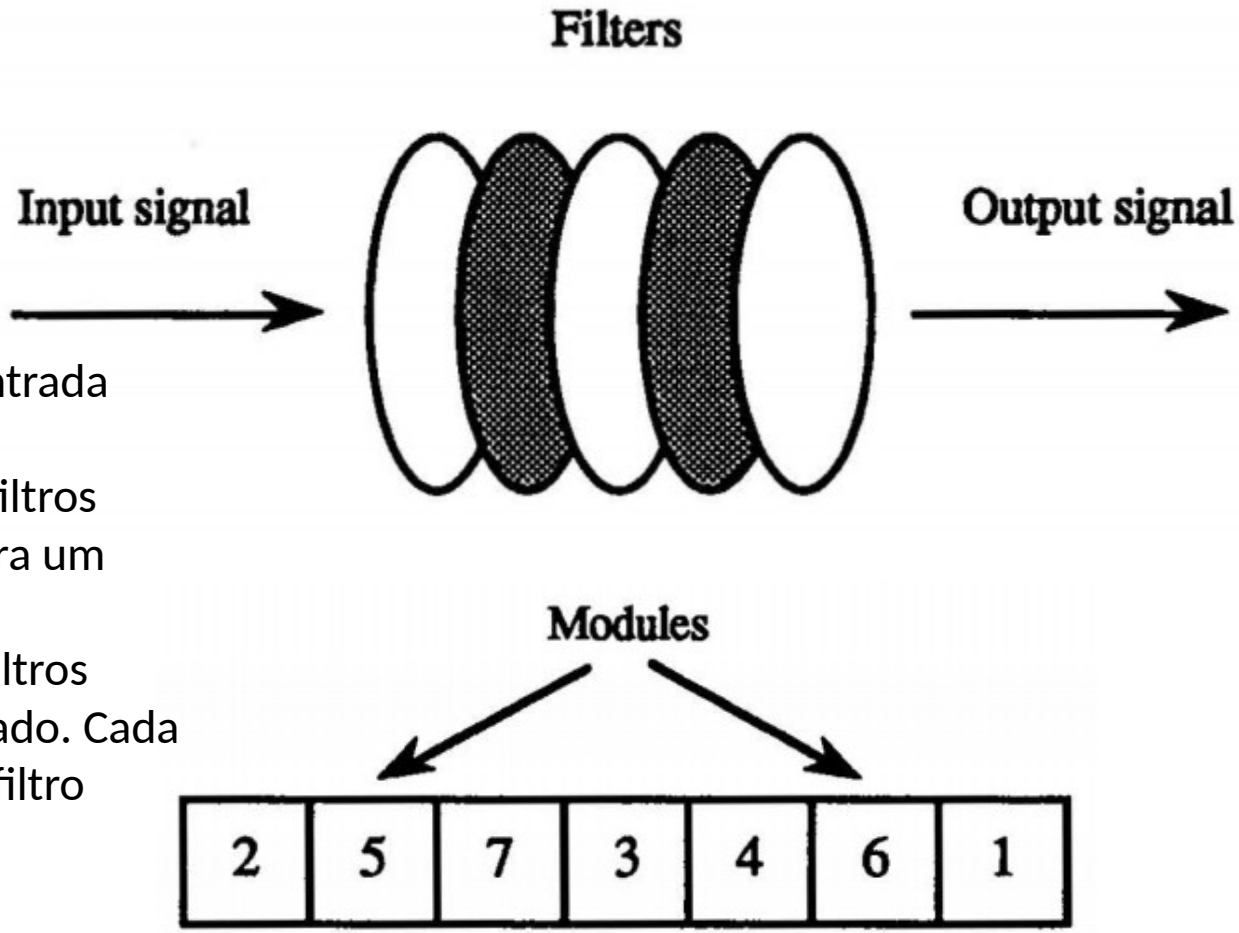
Introdução



Imagine um material composto por camadas (módulos).
Cada módulo tem um efeito de blindagem (isolamento) diferente

Fonte: Glover & Laguna (1997)

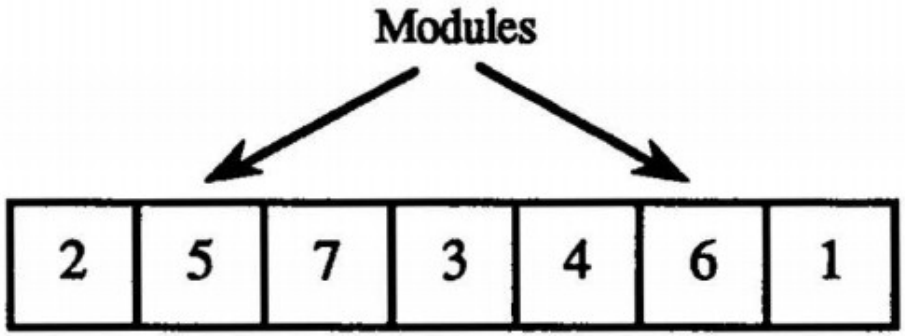
Introdução



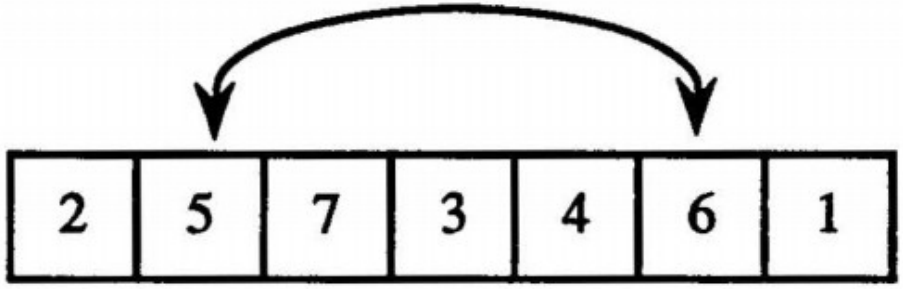
Um sinal de entrada passa por uma sequência de filtros (modules) e vira um sinal de saída.
A ordem dos filtros muda o resultado. Cada número é um filtro diferente.

Fonte: Glover & Laguna (1997)

Introdução



Solução Inicial: 10 unidades de isolamento



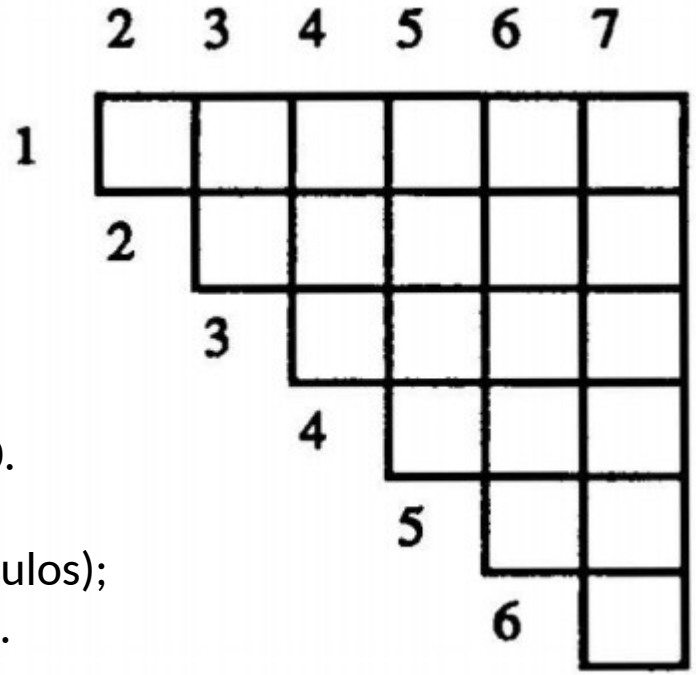
Vizinho

A seta curva indica aplicar um movimento para criar um vizinho (uma permutação “pertinho” da atual).
Insertion (inserção) – tirar um módulo de uma posição e inserir em outra.
Exemplo coerente com a seta: mover o primeiro módulo 2 para o final .
Swap (troca) – trocar as posições de dois módulos.
Exemplo: trocar o primeiro 2 com o último 1 → vizinho

Introdução

2	5	7	3	4	6	1

Solução Inicial: 10



Estrutura Tabu

É o “setup” da Busca Tabu com vizinhança de trocas (swap).
Esquerda: solução inicial [2,5,7,3,4,6,1] com valor 10.
Direita: estrutura tabu em matriz triangular. As linhas/colunas (1...7) representam posições (ou módulos); cada célula (i,j) corresponde ao movimento swap(i,j).
Quando você faz um swap, marca a célula (i,j) com um tenure (n° de iterações restantes). Enquanto não expira, é proibido refazer/“desfazer” esse movimento.
Se um swap tabu gera uma solução melhor que a melhor global, o critério de aspiração permite usá-lo.



Introdução

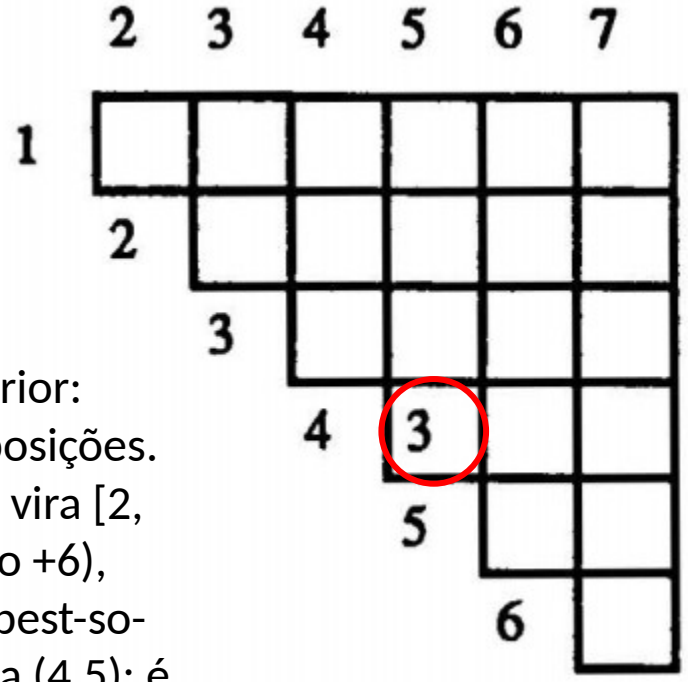
ordene por “Value” e escolha o melhor admissível

Coluna “Swap”: o movimento troca os elementos nas posições indicadas.
ex.: swap(5,4) = trocar o que está nas posições 5 e 4 (ou seja, trocar 4 ↔ 3), gerando [2,5,7,4,3,6,1].
Coluna “Value”: ganho Δ da função-objetivo ao aplicar o swap, em relação ao 10 atual.
6 → novo valor = 10 + 6 = 16
4 → novo valor = 14
2 → 12
0 → 10 (não muda)
−1 → 9 (piora)

Swap		Value	
5	4	6	*
7	4	4	
3	6	2	
2	3	0	
4	1	-1	

Introdução

2	4	7	3	5	6	1



Solução Atual: 16

Escolhemos o melhor movimento da tabela anterior: swap(5,4), aqui o “5” e o “4” são módulos, não posições. Aplicando o swap na ordem [2, 5, 7, 3, 4, 6, 1] → vira [2, 4, 7, 3, 5, 6, 1]. O valor sobe de 10 para 16 (ganho +6), então essa é a solução atual (e possivelmente o best-so-far). Na matriz da direita, colocamos “3” na célula (4,5): é o tenure tabu. Isso proíbe, por 3 iterações, refazer / desfazer esse par de troca (4 ↔ 5). A cada iteração o número decrementa; quando zera, o movimento volta a ser permitido (salvo aspiração).

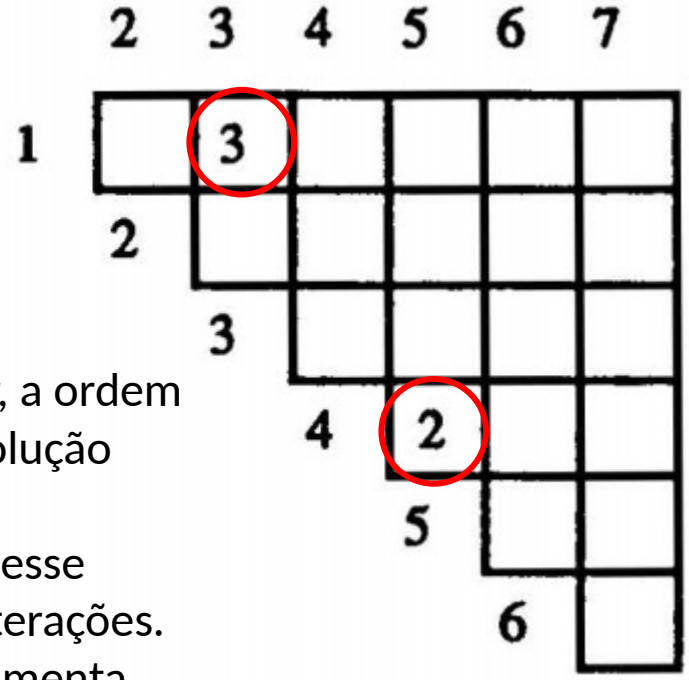
Introdução

Swap		Value	
3	1	2	*
2	3	1	
3	6	-1	
7	1	-2	
6	1	-4	

esse quadro é a lista de movimentos candidatos na 2ª iteração, agora partindo da solução atual [2, 4, 7, 3, 5, 6, 1] com valor 16.

Introdução





Solução Atual: 18

Depois de aplicar swap(3,1) na solução anterior, a ordem vira [2, 4, 7, 1, 5, 6, 3] e o valor sobe para 18 (solução atual/best-so-far).
Na matriz tabu, marcamos o par (1,3) com 3 → esse movimento (e seu inverso) fica proibido por 3 iterações.
O tabu antigo (4,5), criado na 1ª iteração, decrementa para 2 (ainda proibido por 2 iterações).
Próximo passo: gerar vizinhos e escolher o melhor admissível (evitando pares tabu), a menos que um movimento tabu satisfaça aspiração (melhora o melhor global).



Introdução

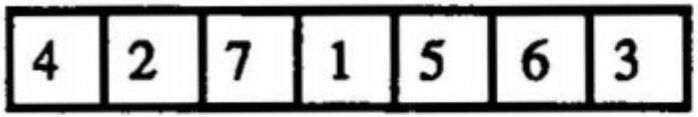
T = tabu no momento (porque já mexemos nesses pares recentemente):
swap(1,3) → −2 (tabu) e swap(4,5) → −7 (tabu).
Como ambos pioram o melhor global (18), não passam por aspiração.
Os admissíveis (não-tabu) são:

swap(2,4) → −4
swap(7,6) → −6
swap(5,3) → −9

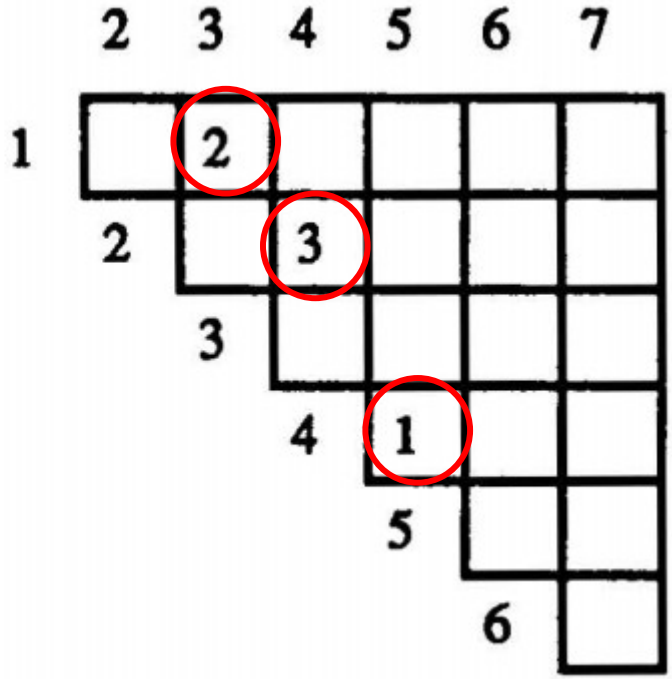
Como não há movimento melhorador, a Busca Tabu segue mesmo assim e escolhe o menos pior admissível (*)(2,4)
Aplicando em [2,4,7,1,5,6,3]→ [4,2,7,1,5,6,3], e o valor cai para 14 (18 − 4).

Swap		Value	
1	3	-2	T
2	4	-4	*
7	6	-6	
4	5	-7	T
5	3	-9	

Introdução



Solução Atual: 14



Introdução

Estamos na solução atual [4,2,7,1,5,6,3], valor 14.
A tabela lista swaps (por módulos) e o ΔValue que cada um traria.
T = movimento tabu no momento. Repare que swap(4,5) está tabu e teria +6 → novo valor seria 20 (14 + 6).
Como 20 > best-so-far = 18, a aspiração libera o tabu (por isso o T*).
Então a escolha correta é swap(4,5) mesmo sendo tabu.

Swap		Value	
4	5	6	T*
5	3	2	
7	1	0	
1	3	-3	T
2	6	-6	

Critério de Aspiração

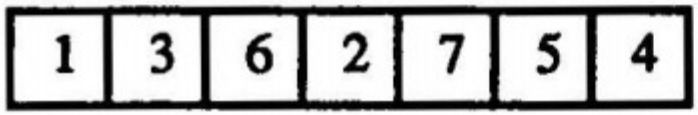
Introdução

5	2	7	1	4	6	3

Solução Atual: 20

	2	3	4	5	6	7
1		1				
2			2			
3						
4				3		
5						
6						

Depois de aplicar a aspiração e fazer swap(4,5), a sequência ficou [5, 2, 7, 1, 4, 6, 3] e o valor subiu para 20 (novo *best-so-far*).
A matriz à direita é a lista tabu com o tempo restante (tenure) de cada par trocado:
(4,5) = 3 → o movimento recém-feito entra tabu por 3 iterações.
(2,4) = 2 → herdado da iteração anterior, decrementou.
(1,3) = 1 → também decrementou.
Enquanto esses contadores não zerarem, não podemos refazer/desfazer esses pares, a menos que o movimento tabu melhore o best-so-far (critério de aspiração).



Insulation Value =12

Solução atual: [1, 3, 6, 2, 7, 5, 4] com valor 12 (o algoritmo já andou bastante e pode estar explorando para sair de um vale).

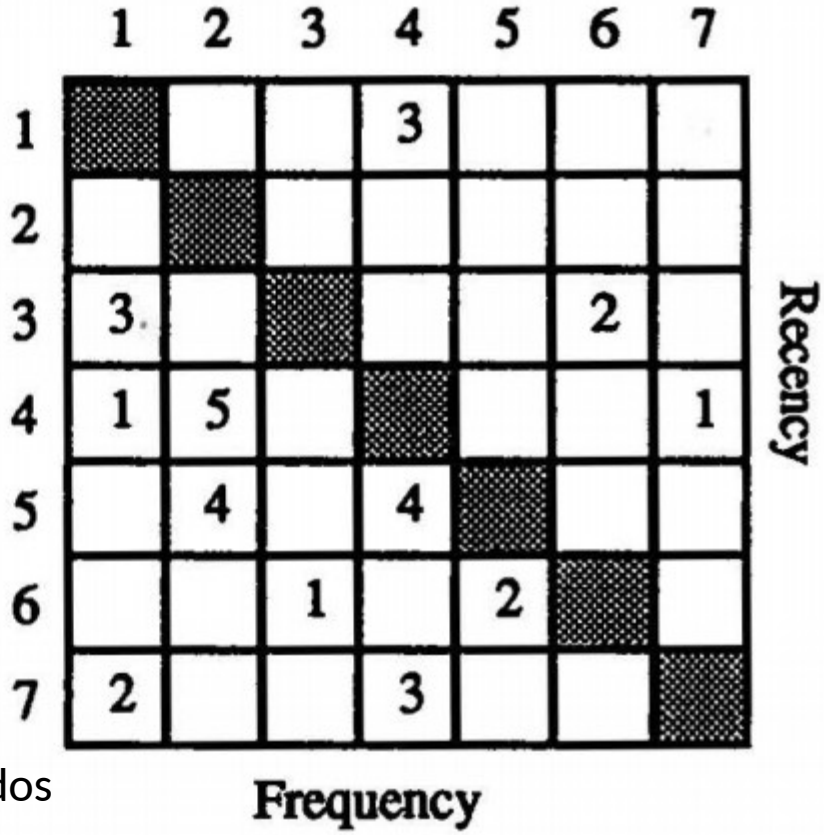
Matriz (triangular): cada célula (i,j) representa o movimento swap(i,j) (trocar os módulos *i* e *j*).

“Frequency” (eixos de baixo/valores nas células): números dentro das células contam quantas vezes aquele swap foi aceito até agora.

→ Servem para diversificação: movimentos muito usados podem receber penalidade (evitar repetir caminhos); raros ganham bônus para explorar regiões novas.

“Recency” (lado direito/sombreados): indica quão recente foi o uso do movimento (ou o tempo restante de tabu).

→ Células sombreadas/indicadas estão tabu por algumas iterações para evitar ciclos; quando o contador zera, voltam a ser permitidas.



26ª Iteração

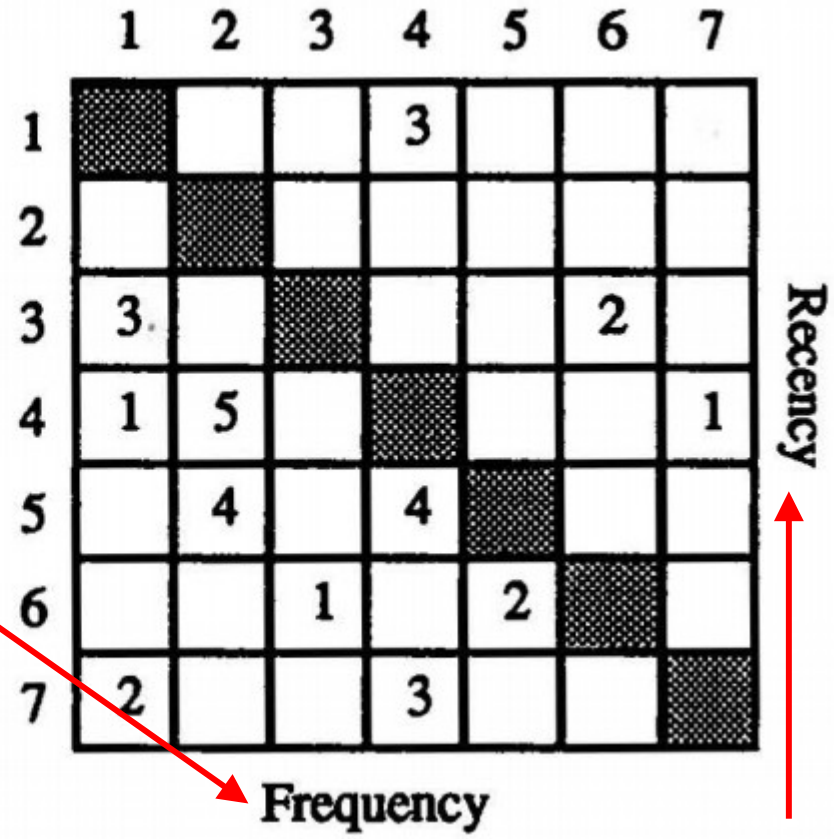
Solução Atual: 12

Introdução

26ª Iteração

1	3	6	2	7	5	4

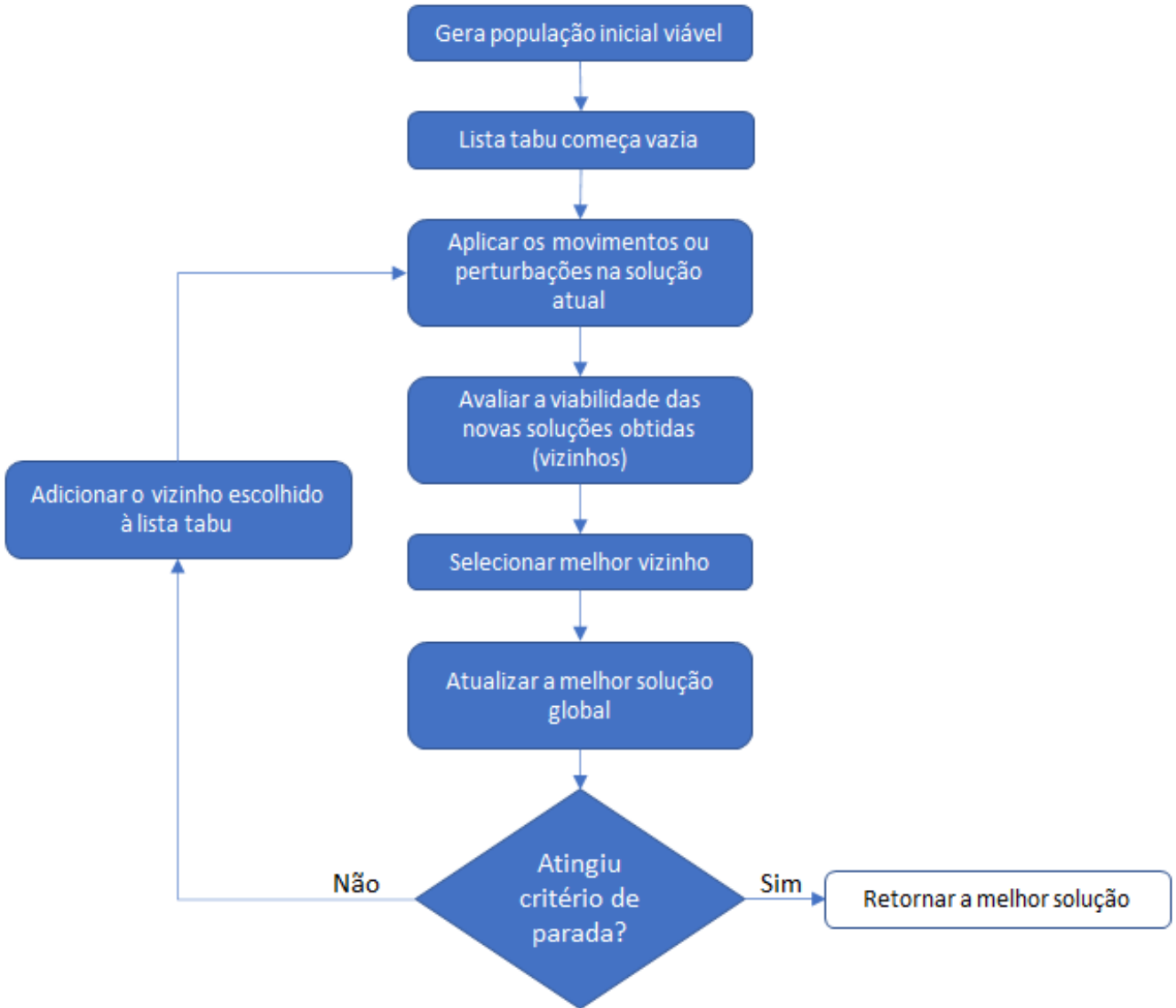
Solução Atual: 12



Memória de longo prazo

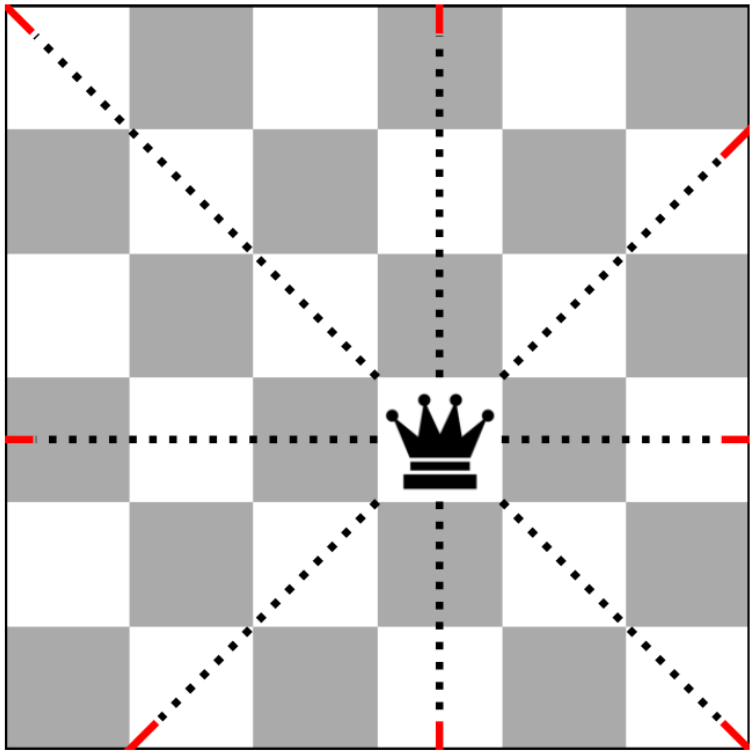
Memória de Curto prazo

Introdução


Gera população inicial viável
Lista tabu começa vazia
Aplicar os movimentos ou perturbações na solução atual
Avaliar a viabilidade das novas soluções obtidas (vizinhos)
Selecionar melhor vizinho
Atualizar a melhor solução global
Atingiu critério de parada?
Não
Sim
Retornar a melhor solução
Adicionar o vizinho escolhido à lista tabu



Problema das 8 Rainhas





Problema das 8 Rainhas

Objetivo: posicionar as 8 rainhas de forma que nenhuma ataque outra

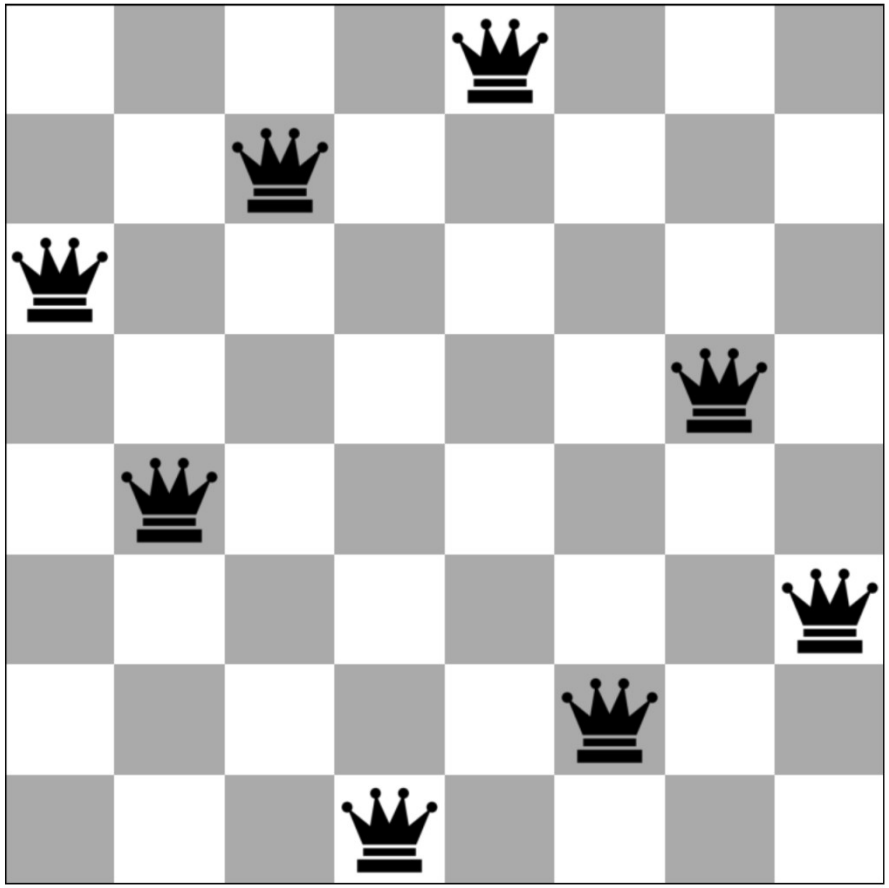
Heurística: número de ataques entre pares de rainhas

Objetivo final: número de ataques = 0

Exploração do espaço de busca dá-se pela mudança de posição de cada uma das rainhas.



Problema das 8 Rainhas





Problema das 8 Rainhas

Solução inicial: cada rainha é posta em uma coluna diferente

Ao longo da execução uma rainha não muda de coluna, apenas de linha

Para cada iteração é permitido então, a cada rainha, sete possíveis movimentos



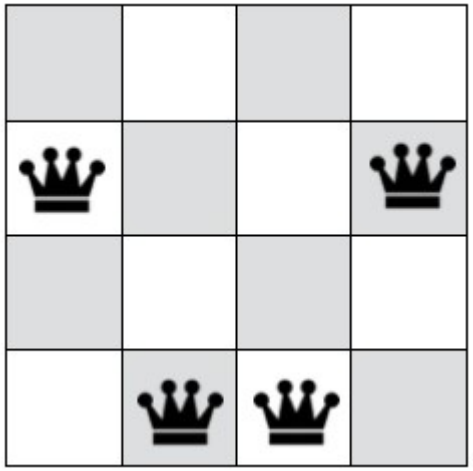
Problema das 8 Rainhas

Simplificando

Regras: colocar 1 rainha por coluna, sem ataques em linhas nem diagonais.

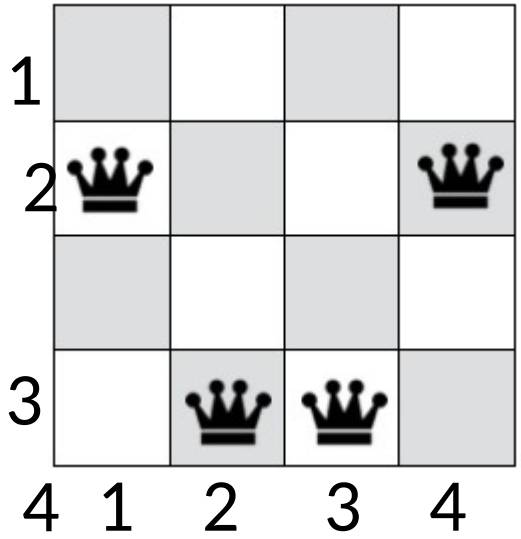
Função-objetivo: número de pares em ataque (queremos 0). No tabuleiro mostrado há ataques porque temos duas rainhas na mesma linha (linha 2) e duas na mesma linha (linha 4), além de diagonais.

Vizinhança (movimento): escolher uma coluna e mover a sua rainha para outra linha. Isso gera os vizinhos que a Busca Tabu avalia; o movimento feito entra na lista tabu por algumas iterações para evitar desfazer imediatamente.



Problema das 8 Rainhas

Simplificando



Vamos usar uma lista tabu de tamanho 1



Problema das 8 Rainhas

O tabuleiro à direita é a configuração atual; o texto “Solução Inicial = 4 ataques” diz que, nesse arranjo, existem 4 conflitos.
A lista de setas (“R1 → R4, R1 → R2, R2 → R3, R2 → R4”) está enumerando vizinhos:
R1 = rainha da coluna 1; movê-la para a linha 4 (ou para a 2).
R2 = rainha da coluna 2; movê-la para a linha 3 (ou para a 4).
(Faria o mesmo para as colunas 3 e 4.)
Para cada candidato, recalcula-se quantos ataques haveria. O “Melhor = 4” indica que nenhum movimento melhora (o melhor vizinho ainda tem 4 conflitos).

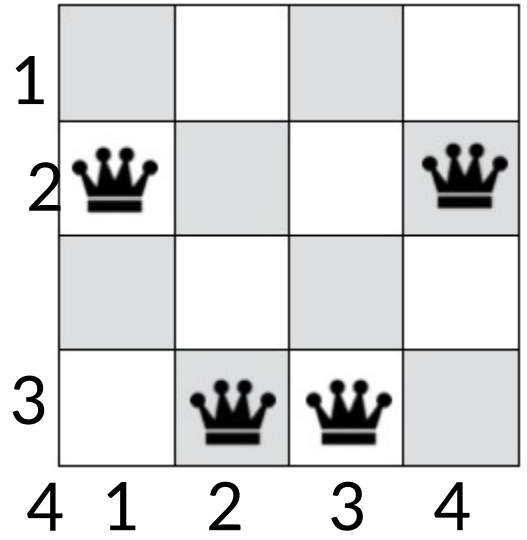
Calculando a solução

R1 → R4

R1 → R2

R2 → R3

R2 → R4



Solução Inicial = 4 ataques

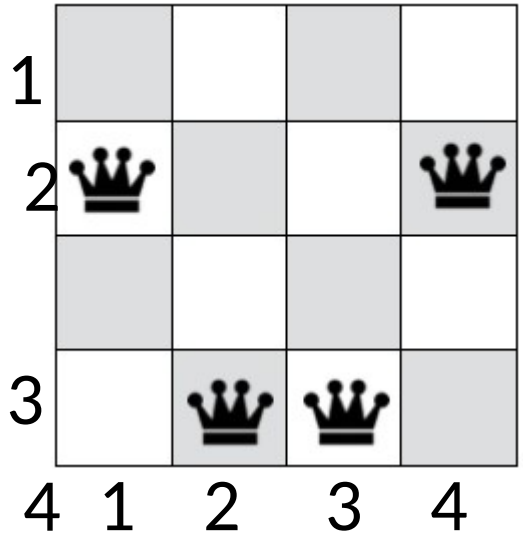
Melhor = 4

Problema das 8 Rainhas

Definindo a vizinhança

- Cada rainha pode se mover para 3 posições diferentes

- Assim, cada solução tem 12 vizinhos



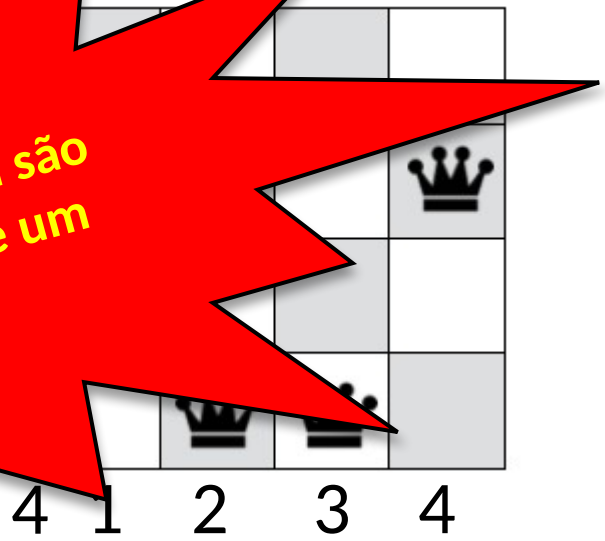
Problema das 8 Rainhas

Definindo a vizinhança

- Cada rainha
  me
  diferen
- Assim,
  12 vi

Importante!

Os vizinhos da solução atual são aqueles obtidos a partir de um único movimento!





Problema das 8 Rainhas

R1 ir para (1,1)

R1 ir para (3,1)

R1 ir para (4,1)

R2 ir para (1,2)

R2 ir para (2,2)

R2 ir para (3,2)

R3 ir para (1,3)

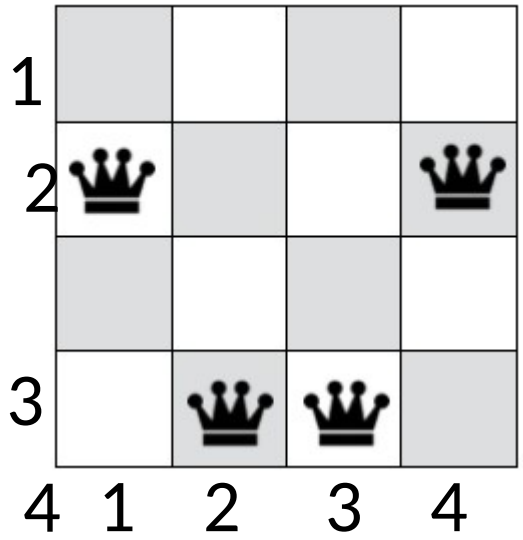
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)



Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1)

R1 ir para (4,1)

R2 ir para (1,2)

R2 ir para (2,2)

R2 ir para (3,2)

R3 ir para (1,3)

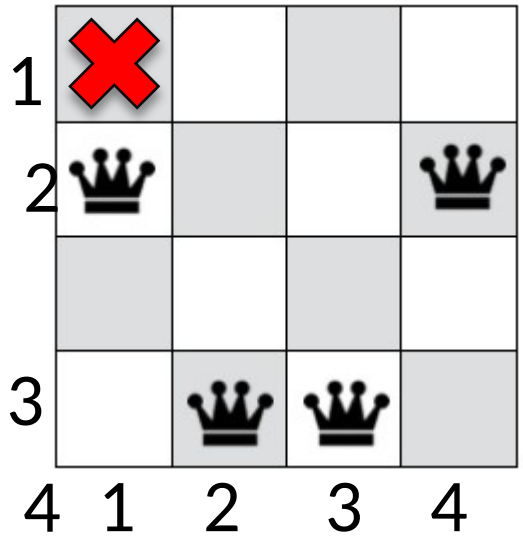
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)



Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1)

R2 ir para (1,2)

R2 ir para (2,2)

R2 ir para (3,2)

R3 ir para (1,3)

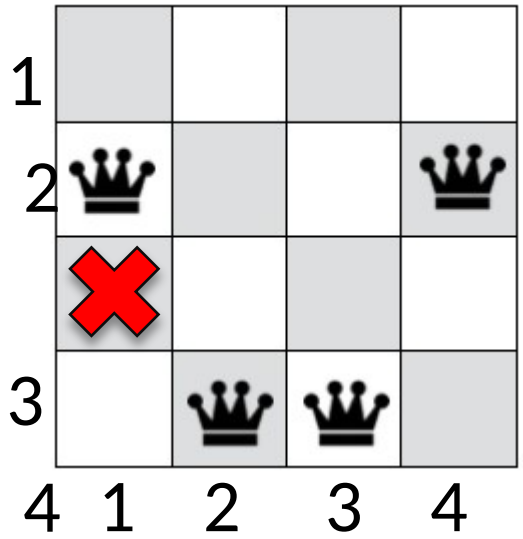
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2)

R2 ir para (2,2)

R2 ir para (3,2)

R3 ir para (1,3)

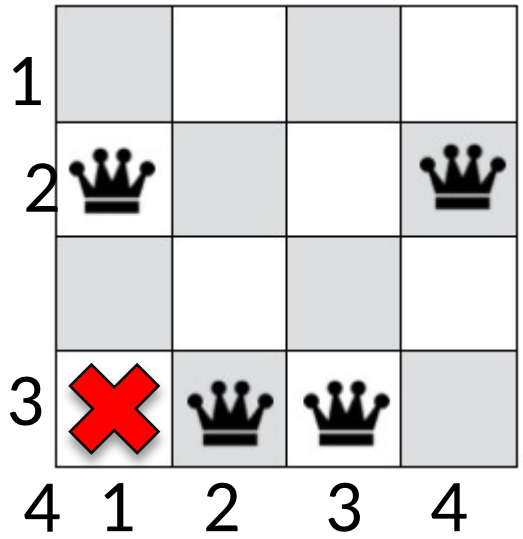
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)



Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2)

R2 ir para (3,2)

R3 ir para (1,3)

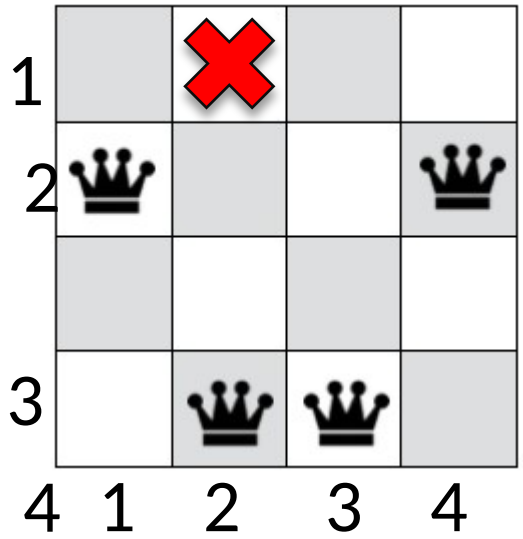
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)



Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2)

R3 ir para (1,3)

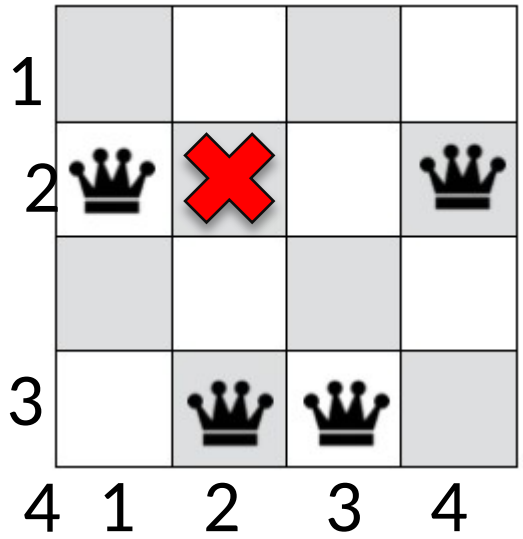
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3)

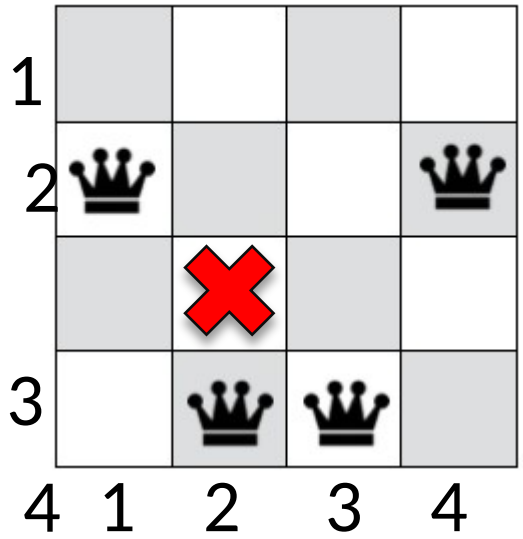
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)



Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

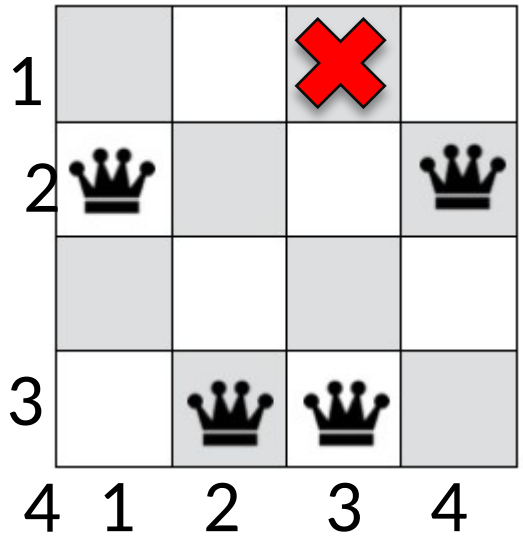
R3 ir para (2,3)

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

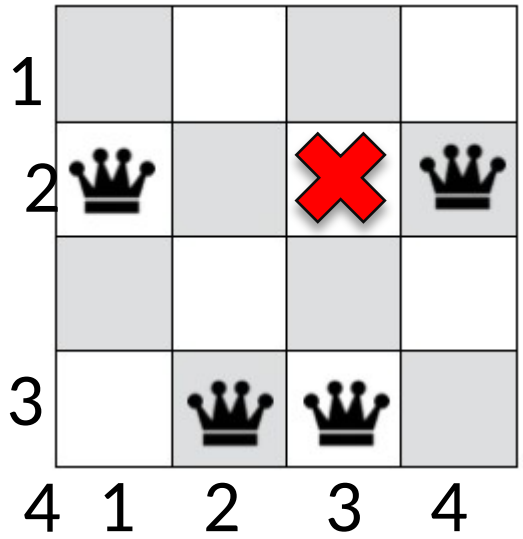
R3 ir para (2,3) = 4 ataques

R3 ir para (3,3)

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

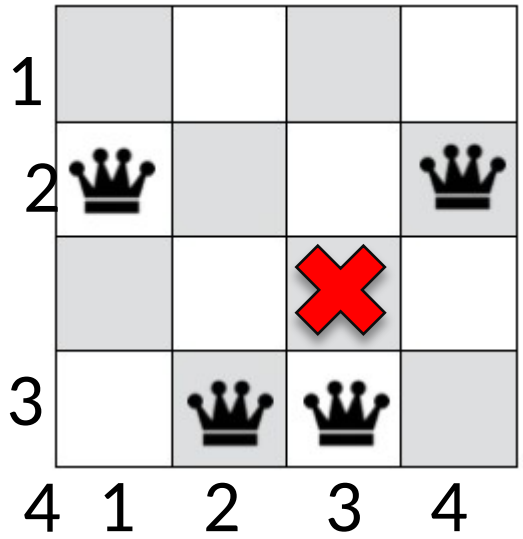
R3 ir para (2,3) = 4 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4)

R4 ir para (3,4)

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

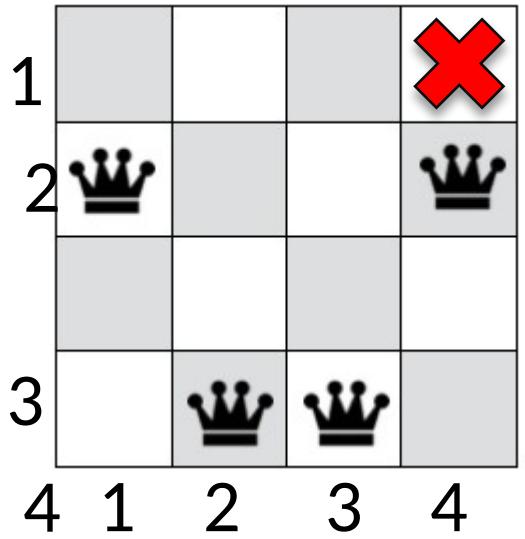
R3 ir para (2,3) = 4 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4) = 2 ataques

R4 ir para (3,4)

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

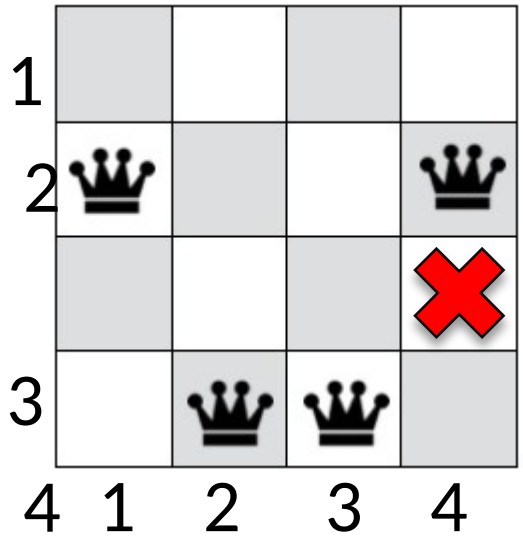
R3 ir para (2,3) = 4 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4) = 2 ataques

R4 ir para (3,4) = 3 ataques

R4 ir para (4,4)





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

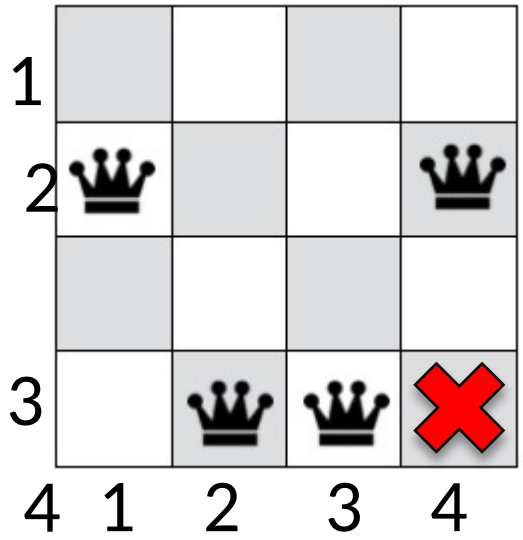
R3 ir para (2,3) = 4 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4) = 2 ataques

R4 ir para (3,4) = 3 ataques

R4 ir para (4,4) = 4 ataques





Problema das 8 Rainhas

R1 ir para (1,1) = 2 ataques

R1 ir para (3,1) = 3 ataques

R1 ir para (4,1) = 4 ataques

R2 ir para (1,2) = 3 ataques

R2 ir para (2,2) = 4 ataques

R2 ir para (3,2) = 4 ataques

R3 ir para (1,3) = 3 ataques

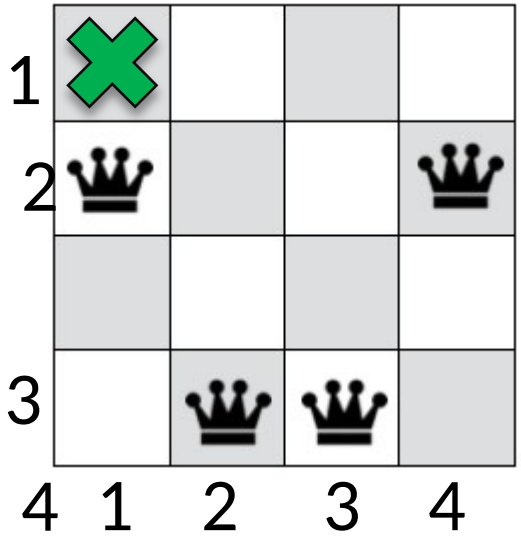
R3 ir para (2,3) = 4 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4) = 2 ataques

R4 ir para (3,4) = 3 ataques

R4 ir para (4,4) = 4 ataques

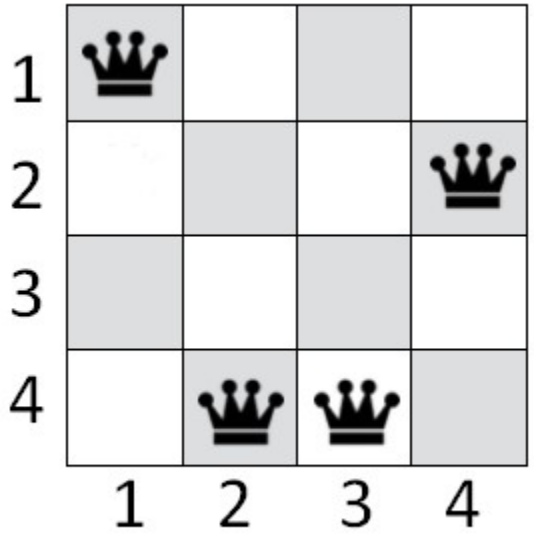


Problema das 8 Rainhas

Essa é então a solução atual $S_1 = 2$

Como o valor obtido é menor que a melhor solução, Melhor = 2

Nesse momento, a rainha R1 é colocada na lista tabu e não pode se movimentar na próxima iteração



Problema das 8 Rainhas

R2 ir para (1,2) = 1 ataque

R2 ir para (2,2) = 2 ataques

R2 ir para (3,2) = 1 ataque

R3 ir para (1,3) = 3 ataques

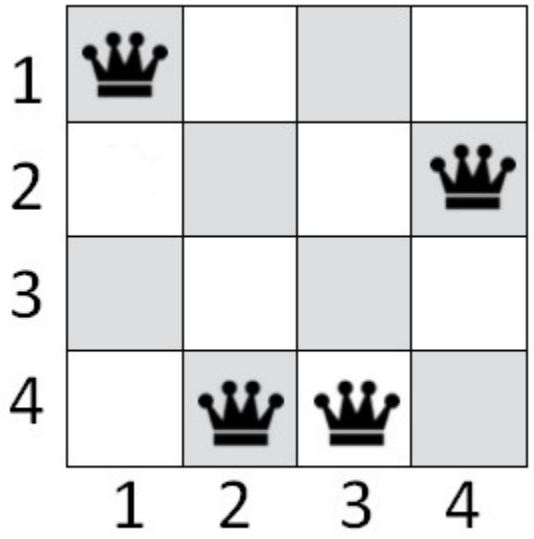
R3 ir para (2,3) = 2 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4) = 2 ataques

R4 ir para (3,4) = 2 ataques

R4 ir para (4,4) = 4 ataques



Problema das 8 Rainhas

R2 ir para (1,2) = 1 ataque

R2 ir para (2,2) = 2 ataques

R2 ir para (3,2) = 1 ataque

R3 ir para (1,3) = 3 ataques

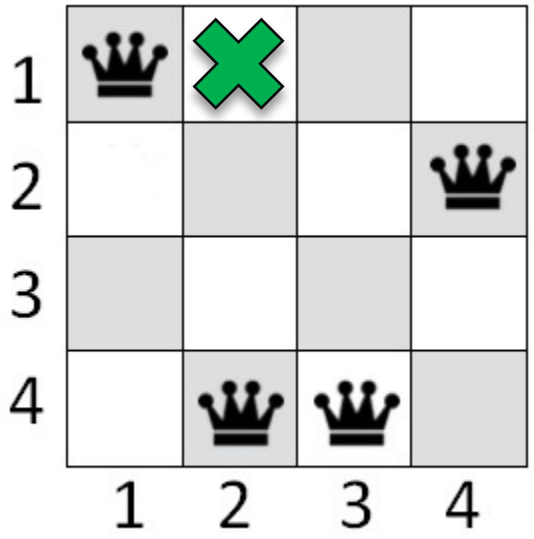
R3 ir para (2,3) = 2 ataques

R3 ir para (3,3) = 4 ataques

R4 ir para (1,4) = 2 ataques

R4 ir para (3,4) = 2 ataques

R4 ir para (4,4) = 4 ataques

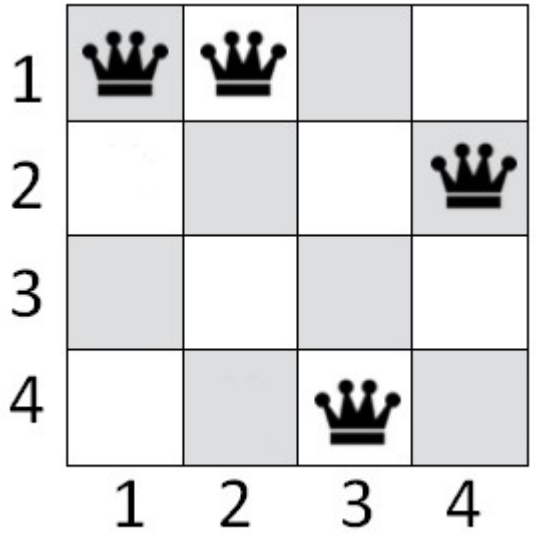


Problema das 8 Rainhas

Essa é então a solução atual $S_2 = 1$

Como o valor obtido é menor que a melhor solução, Melhor = 1

Nesse momento, a rainha R1 tirada da lista tabu e a rainha R2 toma seu lugar na lista



Problema das 8 Rainhas

R1 ir para (2,1) = 3 ataques

R1 ir para (3,1) = 0 ataques

R1 ir para (4,1) = 1 ataque

R3 ir para (1,3) = 4 ataques

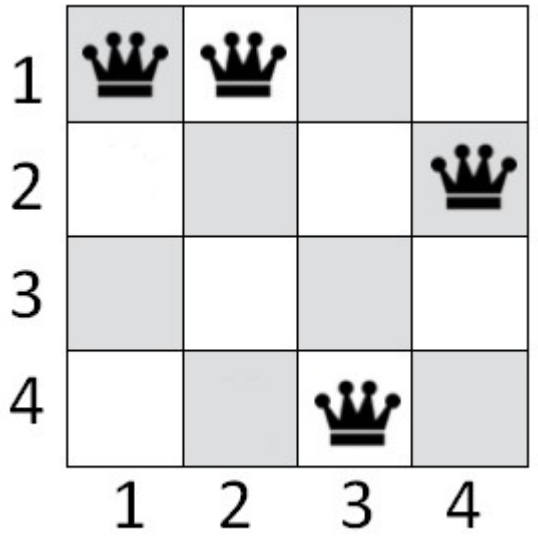
R3 ir para (2,3) = 3 ataques

R3 ir para (3,3) = 3 ataques

R4 ir para (1,4) = 3 ataques

R4 ir para (3,4) = 3 ataques

R4 ir para (4,4) = 3 ataques



Problema das 8 Rainhas

R1 ir para (2,1) = 3 ataques

R1 ir para (3,1) = 0 ataques

R1 ir para (4,1) = 1 ataque

R3 ir para (1,3) = 4 ataques

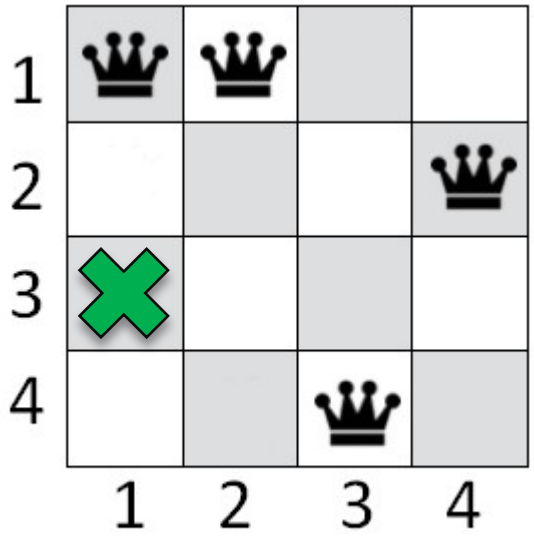
R3 ir para (2,3) = 3 ataques

R3 ir para (3,3) = 3 ataques

R4 ir para (1,4) = 3 ataques

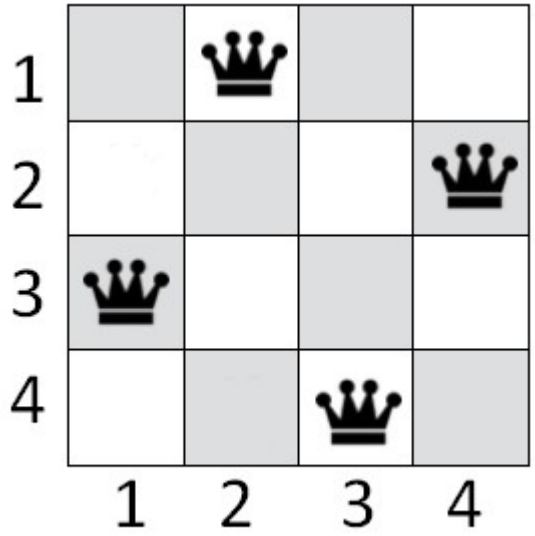
R4 ir para (3,4) = 3 ataques

R4 ir para (4,4) = 3 ataques





Problema das 8 Rainhas









Problema da Mochila Binária

Objetivo

$$\max f(s) = \sum_{j=1}^{n} p_j s_j$$

maximizar o valor total onde p_j é o valor/lucro do item j.

Restrição de Capacidade

Restrição de capacidade: o peso total não pode passar de b. Onde w_j= peso do item j, b= capacidade da mochila.

$$\sum_{j=1}^{n} w_j s_j \leq b$$



Restrição de tipo

$$s_j \in \{0,1\} \; \forall_j = 1..n$$

Decisão: para cada item j, escolha pegar (1) ou não pegar (0) → $s_j \in \{0, 1\}$.



BT aplicada ao Problema da Mochila Binária

Item j	1	2	3	4	5	6	7	8
Peso w_j	4	15	7	9	8	10	9	11
Retorno p_j	2	2	3	4	6	5	8	7

Capacidade da Mochila, $b = 33$

$s = \{s_1, s_2, .., s_n\}$ em que $s_j \in \{0,1\}$

Uma solução vizinha à s será qualquer sequência s com um bit modificado, formando assim um conjunto s' composto por n elementos.



BT aplicada ao Problema da Mochila Binária

Item j	1	2	3	4	5	6	7	8
Peso w_j	4	15	7	9	8	10	9	11
Retorno p_j	2	2	3	4	6	5	8	7

Vamos considerar a solução S^0 a seguir:

$$S^0 = (1,0,0,1,0,1,1,0) \mid \text{itens 1,4,6,7 escolhidos}$$

$$f(S^0) = 19$$

$$peso(S^0) = 32$$

Para encontrarmos um vizinho de S^0 pode-se inverter qualquer um dos bits que compõe a solução. Nesse caso, $|V| = 8$



BT aplicada ao Problema da Mochila Binária

S_0	1	0	0	1	0	1	1	0	32	19

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**0**	0	0	1	0	1	1	0	28	17
2	1	**1**	0	1	0	1	1	0	47	-
3	1	0	**1**	1	0	1	1	0	39	-
4	1	0	0	**0**	0	1	1	0	23	15
5	1	0	0	1	**1**	1	1	0	40	-
6	1	0	0	1	0	**0**	1	0	22	14
7	1	0	0	1	0	1	**0**	0	23	11
8	1	0	0	1	0	1	1	**1**	43	-

F(s') = retorno



BT aplicada ao Problema da Mochila Binária

A solução inicial representa um ótimo local, uma vez que seu benefício mostrou-se maior que todos os seus vizinhos

A Busca Tabu, no entanto, aceita escolhas de não-melhora como estratégia para tentar encontrar uma solução ótima global



BT aplicada ao Problema da Mochila Binária

Lista Tabu T

S_0	1	0	0	1	0	1	1	0	32	19

T elementos

Quando a tabela alcançar o tamanho T, a solução mais antiga será substituída pela mais recente

Quanto maior for |T| mais longa é a proibição de volta a uma solução já visitada

Quanto menor for |T| menor a restrição de revisita



BT aplicada ao Problema da Mochila Binária

Lista Tabu T

S_0	1	0	0	1	0	1	1	0	32	19

Ao invés de armazenar a solução inteira, pode-se armazenar apenas a alteração que leva à solução tabu. Nesse caso, as inversões binárias...

ST = {<1>, < >} → considerando |T| = 2



BT aplicada ao Problema da Mochila Binária

S_1	0	0	0	1	0	1	1	0	28	17

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**1**	0	0	1	0	1	1	0	32	19
2	0	**1**	0	1	0	1	1	0	43	-
3	0	0	**1**	1	0	1	1	0	35	-
4	0	0	0	**0**	0	1	1	0	19	13
5	0	0	0	1	**1**	1	1	0	36	-
6	0	0	0	1	0	**0**	1	0	18	12
7	0	0	0	1	0	1	**0**	0	19	9
8	0	0	0	1	0	1	1	**1**	39	-



BT aplicada ao Problema da Mochila Binária

S_1	0	0	0	1	0	1	1	0	28	17

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**1**	0	0	1	0	1	1	0	32	19
2	0	**1**	0	1	0	1	1	0	43	-
3	0	0	**1**	1	0	1	1	0	35	-
4	0	0	0	**0**	0	1	1	0	19	13
5	0	0	0	1	**1**	1	1	0	36	-
6	0	0	0	1	0	**0**	1	0	18	12
7	0	0	0	1	0	1	**0**	0	19	9
8	0	0	0	1	0	1	1	**1**	39	-



BT aplicada ao Problema da Mochila Binária

Lista Tabu T

S_1	0	0	0	1	0	1	1	0	28	17

S_2	0	0	0	0	0	1	1	0	19	13

Nesse momento a segunda solução é adicionada à lista tabu

ST = {<1>, <4>}



BT aplicada ao Problema da Mochila Binária

S_2	0	0	0	0	0	1	1	0	19	13

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**1**	0	0	0	0	1	1	0	23	15
2	0	**1**	0	0	0	1	1	0	34	-
3	0	0	**1**	0	0	1	1	0	26	16
4	0	0	0	**1**	0	1	1	0	28	17
5	0	0	0	0	**1**	1	1	0	27	19
6	0	0	0	0	0	**0**	1	0	9	8
7	0	0	0	0	0	1	**0**	0	10	5
8	0	0	0	0	0	1	1	**1**	30	20



BT aplicada ao Problema da Mochila Binária

Lista Tabu T

S_2	0	0	0	0	0	1	1	0	19	13

S_3	0	0	0	0	0	1	1	1	30	20

Nesse momento o primeiro bit deixa de ser tabu e o oitavo bit é adicionado à lista tabu

ST = {<4>, <8>}



BT aplicada ao Problema da Mochila Binária

S_3	0	0	0	0	0	1	1	1	30	20

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**1**	0	0	0	0	1	1	1	34	-
2	0	**1**	0	0	0	1	1	1	45	-
3	0	0	**1**	0	0	1	1	1	37	-
4	0	0	0	**1**	0	1	1	1	39	-
5	0	0	0	0	**1**	1	1	1	38	-
6	0	0	0	0	0	**0**	1	1	20	15
7	0	0	0	0	0	1	**0**	1	21	12
8	0	0	0	0	0	1	1	**0**	19	13

ST = {<8>, <6>}



BT aplicada ao Problema da Mochila Binária

S_4	0	0	0	0	0	0	1	1	20	15

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**1**	0	0	0	0	0	1	1	24	17
2	0	**1**	0	0	0	0	1	1	35	-
3	0	0	**1**	0	0	0	1	1	27	18
4	0	0	0	**1**	0	0	1	1	29	19
5	0	0	0	0	**1**	0	1	1	28	21
6	0	0	0	0	0	**1**	1	1	30	20
7	0	0	0	0	0	0	**0**	1	11	7
8	0	0	0	0	0	0	1	**0**	9	8

ST = {<6>, <5>}



BT aplicada ao Problema da Mochila Binária

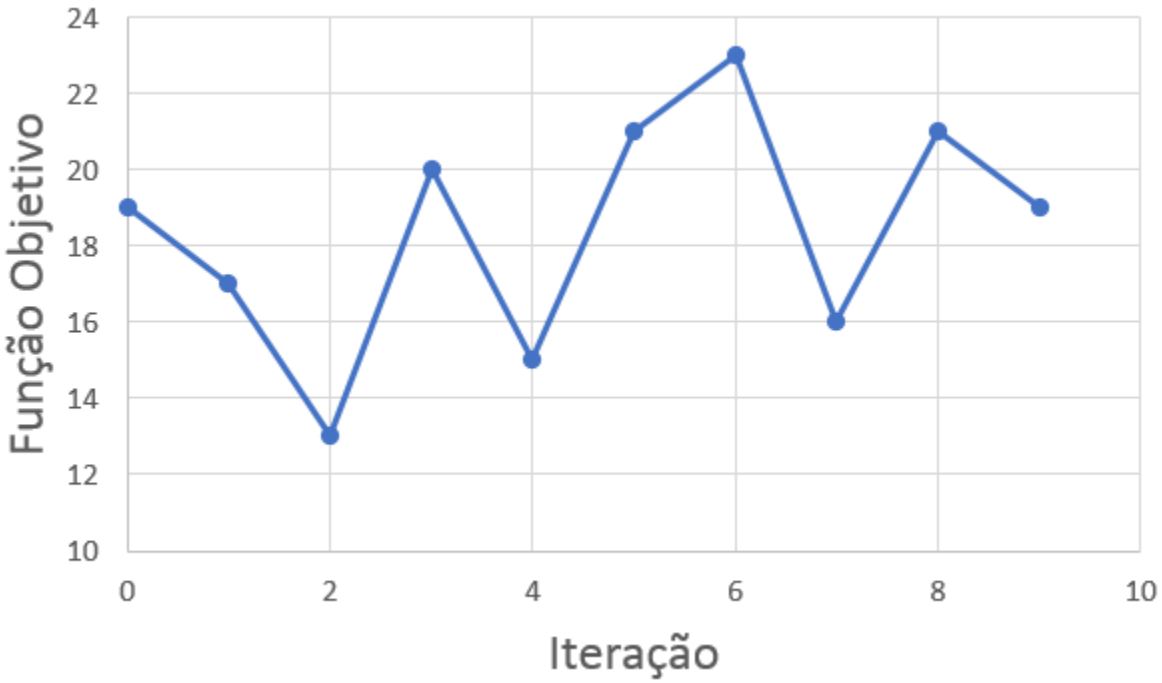
S_5	0	0	0	0	1	0	1	1	28	21

Vizinhança	1	2	3	4	5	6	7	8	Peso(s')	f(s')
1	**0**	0	0	0	1	0	1	1	32	23
2	0	**0**	0	0	1	0	1	1	43	-
3	0	0	**0**	0	1	0	1	1	35	-
4	0	0	0	**0**	1	0	1	1	37	-
5	0	0	0	0	**1**	0	1	1	20	15
6	0	0	0	0	1	**0**	1	1	38	-
7	0	0	0	0	1	0	**1**	1	19	13
8	0	0	0	0	1	0	1	**1**	17	14

ST = {<5>, <1>}

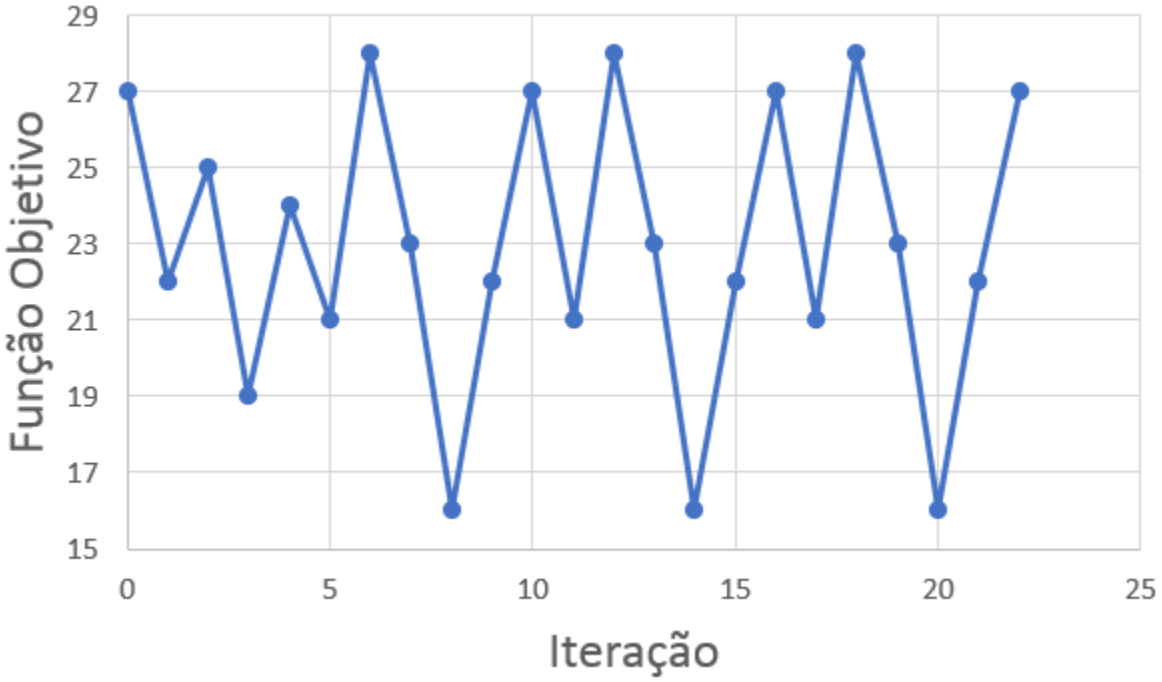


BT aplicada ao Problema da Mochila Binária





BT aplicada ao Problema da Mochila Binária





Exercício

Capacidade $b = 10$ $(peso)$. Valor é livre.

Escolha itens para maximizar o valor.

Item 1 — peso 2, valor 3

Item 2 — peso 3, valor 4

Item 3 — peso 4, valor 5

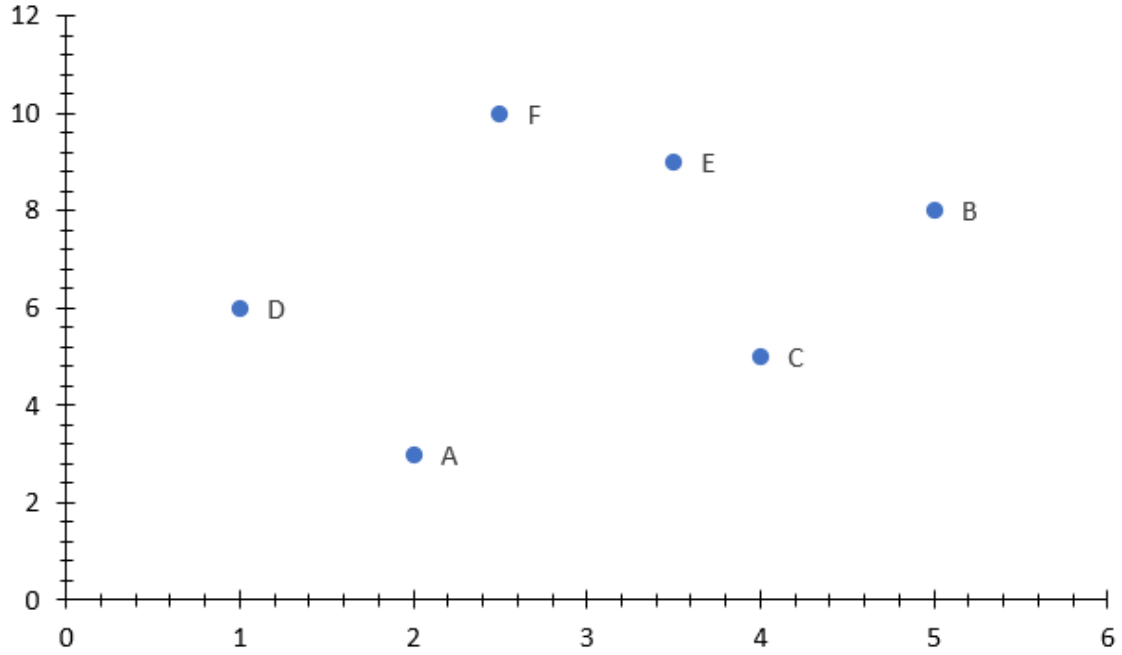
Item 4 — peso 5, valor 8

Item 5 — peso 9, valor 10

Listar combinações que caibam e achar a de maior valor; escrever também o vetor binário s.



Problema do Caixeiro Viajante TSP





Problema do Caixeiro Viajante TSP

	A	B	C	D	E
A	0				
B	5	0			
C	2	4	0		
D	7	3	8	0	
E	6	9	2	10	0



Problema da Designação Generalizada - PDG

Problema:

- Um conjunto de atividades deve ser realizada por um grupo de agentes.
- Todas as atividades devem ser realizadas de forma a minimizar o custo de realização das mesmas (Função Objetivo)
- Assim, cada tarefa tem um determinado custo de execução
- Além do custo de execução, cada tarefa tem uma quantidade de recurso atrelada à sua execução (limitado pelas capacidades/disponibilidades dos agentes)



Problema da Designação Generalizada - PDG

Custos para execução de cada atividade por cada agente

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	23	38	33	34	26	19	12	37
A2	36	45	13	41	14	25	49	16
A3	17	15	30	40	29	27	47	22

A tabela mostra o custo que cada agente teria para executar cada tarefa.
Linhas: representam os agentes (A1, A2, A3).
Colunas: representam as tarefas (T1 até T8).
Cada número indica o custo associado.
O desafio do PDG é encontrar uma forma de distribuir todas as tarefas entre os agentes, respeitando restrições (como limite de recursos) e alcançando o menor custo total possível.



Problema da Designação Generalizada - PDG

Quantidade de recurso usado por cada agente para realizar as tarefas

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	21	13	4	3	7	15	22	24
A2	20	8	23	25	6	5	9	14
A3	16	12	18	2	11	10	17	19



Problema da Designação Generalizada - PDG

Quantidade de recurso disponível por cada agente

$$A_1 = 30$$
$$A_2 = 25$$
$$A_3 = 35$$



PDG – Abordagem Construtiva

Passo 1	**Designar T_3 a A_1: custo de 33 e consumo de recurso de 4**															
T_3-A_1	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	23	38	33	34	26	19	12	37	21	13	4	3	7	15	22	24
A_2	36	45	13	41	14	25	49	16	20	8	23	25	6	5	9	14
A_3	17	15	30	40	29	27	47	22	16	12	18	2	11	10	17	19
Arrependimento do consumo de recursos:									4	4	14	1	1	5	8	5

Disponibilidade dos agentes: [30, 25, 35] - [4, 0, 0] = [26, 25, 35]



PDG – Abordagem Construtiva

Passo 2	**Designar T_7 a A_2: custo de 49 e consumo de recurso de 9**															
T_7-A_2	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	23	38	*	34	26	19	12	37	21	13	*	3	7	15	22	24
A_2	36	45	*	41	14	25	49	16	20	8	*	25	6	5	9	14
A_3	17	15	*	40	29	27	47	22	16	12	*	2	11	10	17	19
Arrependimento do consumo de recursos:									4	4	*	1	1	5	8	5
Disponibilidade dos agentes: [26, 25, 35] - [0, 9, 0] = [26, 16, 35]																

As colunas de T3 aparecem com *, pois já foram atribuídas no Passo 1.
Arrependimento: Quando designamos uma tarefa a um agente, ela consome uma certa quantidade de recurso desse agente.
Mas, se tivéssemos escolhido outro agente para a mesma tarefa, o consumo poderia ter sido menor. O arrependimento mede exatamente essa diferença: é o “custo extra” em termos de recursos por não ter feito a escolha mais econômica.



PDG – Abordagem Construtiva

Passo 3	**Designar T_6 a A_2: custo de 25 e consumo de recurso de 5**															
T_6-A_2	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	23	38	*	34	26	19	*	37	21	13	*	3	7	15	*	24
A_2	*	45	*	*	14	25	*	16	*	8	*	*	6	5	*	14
A_3	17	15	*	40	29	27	*	22	16	12	*	2	11	10	*	19
Arrependimento do consumo de recursos:									5	4	*	1	1	5	*	5

Disponibilidade dos agentes: [26, 16, 35] - [0, 5, 0] = [26, 11, 35]



PDG – Abordagem Construtiva

Passo 4	**Designar T_1 a A_3: custo de 17 e consumo de recurso de 16**															
T_1-A_3	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	23	38	*	34	26	*	*	37	21	13	*	3	7	*	*	24
A_2	*	45	*	*	14	*	*	*	*	8	*	*	6	*	*	*
A_3	17	15	*	40	29	*	*	22	16	12	*	2	11	*	*	19
Arrependimento do consumo de recursos:									5	4	*	1	1	*	*	5

Disponibilidade dos agentes: [26, 11, 35] - [0, 0, 16] = [26, 11, 19]



PDG – Abordagem Construtiva

Passo 5	**Designar T_8 a A_3: custo de 22 e consumo de recurso de 19**															
T_8-A_3	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	*	38	*	34	26	*	*	37	*	13	*	3	7	*	*	24
A_2	*	45	*	*	14	*	*	*	*	8	*	*	6	*	*	*
A_3	*	15	*	40	29	*	*	22	*	12	*	2	11	*	*	19
Arrependimento do consumo de recursos:									*	4	*	1	1	*	*	5

Disponibilidade dos agentes: [26, 11, 19] - [0, 0, 19] = [26, 11, 0]



PDG – Abordagem Construtiva

Passo 6	**Designar T_4 a A_1: custo de 34 e consumo de recurso de 3**															
T_4-A_1	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	*	38	*	34	26	*	*	*	*	13	*	3	7	*	*	*
A_2	*	45	*	*	14	*	*	*	*	8	*	*	6	*	*	*
A_3	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
Arrependimento do consumo de recursos:									*	5	*	MAX	1	*	*	*

Disponibilidade dos agentes: [26, 11, 0] - [3, 0, 0] = [23, 11, 0]



PDG – Abordagem Construtiva

Passo 7	**Designar T_2 a A_2: custo de 45 e consumo de recurso de 8**															
T_2-A_2	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	*	38	*	*	26	*	*	*	*	13	*	*	7	*	*	*
A_2	*	45	*	*	14	*	*	*	*	8	*	*	6	*	*	*
A_3	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
Arrependimento do consumo de recursos:									*	5	*	*	1	*	*	*

Disponibilidade dos agentes: [23, 11, 0] - [0, 8, 0] = [23, 3, 0]



PDG – Abordagem Construtiva

Passo 8	**Designar T_5 a A_1: custo de 26 e consumo de recurso de 7**															
T_5-A_1	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	*	*	*	*	26	*	*	*	*	*	*	*	7	*	*	*
A_2	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
A_3	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
Arrependimento do consumo de recursos:									*	*	*	*	MAX	*	*	*

Disponibilidade dos agentes: [23, 3, 0] - [7, 0, 0] = [16, 3, 0]



PDG – Abordagem Construtiva

		Capacidade dos agentes			
Agente	Tarefas	Inicial	Utilizada	Final (disponível)	Custo
A_1	T_3 T_4 T_5	30	14	16	93
A_2	T_2 T_6 T_7	25	22	3	119
A_3	T_1 T_8	35	35	0	39
				Custo Total:	**251**



PDG – Abordagem Construtiva

Custos

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	23	38	33	34	26	19	12	37
A2	36	45	13	41	14	25	49	16
A3	17	15	30	40	29	27	47	22

Consumo de recursos

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	21	13	4	3	7	15	22	24
A2	20	8	23	25	6	5	9	14
A3	16	12	18	2	11	10	17	19



PDG – Abordagem Construtiva

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	23	38	33	34	26	19	12	37
A2	36	45	13	41	14	25	49	16
A3	17	15	30	40	29	27	47	22

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	21	13	4	3	7	15	22	24
A2	20	8	23	25	6	5	9	14
A3	16	12	18	2	11	10	17	19



PDG – Abordagem Construtiva

		Capacidade dos agentes			
Agente	Tarefas	Inicial	Utilizada	Final (disponível)	Custo
A_1	T_3 T_8	30	28	2	70
A_2	T_2 T_5 T_6	25	19	6	84
A_3	T_1 T_4 T_7	35	35	0	104
				Custo Total:	**258**

Esta solução inicial difere da anterior. No entanto, ela está sendo usada como base pois permite uma melhor percepção da evolução do algoritmo



PDG – Abordagem Construtiva

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	23	38	33	34	26	19	12	37
A2	36	45	13	41	14	25	49	16
A3	17	15	30	40	29	27	47	22

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	21	13	4	3	7	15	22	24
A2	20	8	23	25	6	5	9	14
A3	16	12	18	2	11	10	17	19



PDG – Busca Tabu

Movimentação (perturbação) para definir a vizinhança é do tipo inserção: uma tarefa j de um agente r é atribuída a outro agente s



Duração tabu: 3 iterações (a tarefa j atribuída ao agente s deve permanecer com ele por três iterações)

Ao realizar a movimentação um vizinho pode ser uma solução factível (a capacidade de nenhum vizinho é excedida) ou infactível (pelo menos um vizinho tem sua capacidade excedida)



PDG – Busca Tabu

Penalização

Ao impedir que soluções infactíveis sejam avaliadas o conjunto de possibilidades pode diminuir consideravelmente. Uma estratégia para permitir tais soluções é a penalidade.

A penalização é realizada pela multiplicação da quantidade total que excede a capacidade dos agentes por um parâmetro positivo P. Assim, esse valor é adicionado à função objetivo da solução de forma a penalizá-la por não cumprir as.



PDG – Busca Tabu

Penalização

Valores muito alto para P fazem com que a busca priorize movimentos de baixa ou nenhuma infactibilidade.

Valores muito baixos podem levar a um conjunto grande de soluções infactíveis.

Nesse exemplo será adotado um P=2



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_8	2	0	70	0	70	-19	-11	•	-1	-5	-13	-20	•
A_2	T_2 T_5 T_6	6	0	84	0	84	-14	•	-17	-19	•	•	-3	-8
A_3	T_1 T_4 T_7	0	0	104	0	104	•	-12	-18	•	-11	-10	•	-19
		Custo Total:		**258**	**0**	**258**								

Cada agente (A1, A2, A3) tem suas tarefas atribuídas, os custos calculados e a capacidade utilizada.

O custo total inicial é 258, sem excesso de recursos.

A tabela da direita (“Cap. disp. após movimento”) indica como ficaria a disponibilidade de recursos se cada tarefa fosse movida para outro agente.

Os valores negativos mostram que a mudança causaria estouro de capacidade, logo seriam movimentos inviáveis, a menos que a busca tabu permita pela regra de aspiração.



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_8	2	0	70	0	70	-19	-11	•	-1	-5	-13	-20	•
A_2	T_2 T_5 T_6	6	0	84	0	84	-14	•	-17	-19	•	•	-3	-8
A_3	T_1 T_4 T_7	0	0	104	0	104	•	-12	-18	•	-11	-10	•	-19
		Custo Total:		**258**	**0**	**258**								

Para calcular as soluções vizinhas e selecionar a melhor solução, o custo do movimento de inserção é calculado somando a variação do custo de alocação (custo final – custo inicial) ao custo da infactibilidade (valor de infactibilidade * P)

PDG – Busca Tabu

	Tarefa							
Agente	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	44	15	•	-4	22	20	5	•
A_2	47	•	14	39	•	•	8	-5
A_3	•	-6	33	•	37	22	•	23

Por exemplo, para avaliar a atribuição da Tarefa T_4 (que está com o agente A_3) para o agente A_1

Custo Final – Custo Inicial + (Infactibilidade * P)

Custo A_1T_4 – Custo A_3T_4 + ((Demanda A_1T_4 – Disponibilidade de A_1) *2)

34 – 40 + ((3-2)*2) = -4



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_8	2	0	70	0	70	-19	-11	•	-1	-5	-13	-20	•
A_2	T_5 T_6	14	0	39	0	39	-6	6	-9	-11	•	•	5	0
A_3	T_1 T_4 T_7 T_2	0	12	119	24	143	•	•	-18	•	-11	-10	•	-19
		Custo Total:		**228**	**24**	**252**								

Na solução inicial o custo total era 258.
Após os movimentos da Busca Tabu, o custo de alocação caiu para 228.
Porém, houve excesso de recursos (12 unidades no A3), o que gera uma penalidade de 24.
Assim, o custo total ajustado ficou em 252.
A Busca Tabu conseguiu reduzir o custo de execução (de 258 → 228).
Porém, devido à infactibilidade (excesso de recursos no A3), o custo ajustado sobe para 252.
Ainda assim, a solução é melhor que a inicial (258), mostrando como a Busca Tabu encontra alternativas que exploram além do ótimo local, mesmo aceitando violações temporárias das restrições.

PDG – Busca Tabu

	Tarefa							
Agente	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	20	21	•	-8	22	20	-19	•
A_2	7	6	-2	19	•	•	-22	-21
A_3	•	🚫	33	•	37	22	•	23

Cada célula indica o impacto de mover uma tarefa para um determinado agente.
Valores positivos → aumentam o custo.
Valores negativos → representam melhoria (redução de custo).
O símbolo 🚫 indica que esse movimento está proibido pela lista Tabu (movimento recente ou revertido, que não pode ser refeito por algumas iterações).
Se a T2 for movida para o agente A1, o custo aumenta em +21.
Se a T4 for movida para o agente A1, há uma redução de -8.
Se a T7 for movida para o agente A2, o custo cairia em -22 (ótima melhoria).



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_8	2	0	70	0	70	-19	-11	•	-1	-5	-13	-20	•
A_2	T_5 T_6 T_7	5	0	88	0	88	-15	-3	-18	-20	•	•	•	-9
A_3	T_1 T_4 T_2	5	0	72	0	72	•	•	-13	•	-6	-5	-12	-14
	Custo Total:			**230**	**0**	**230**								

A1 ficou com as tarefas T3 e T8, custo total = 70.
A2 ficou com as tarefas T5, T6 e T7, custo total = 88.
A3 ficou com as tarefas T1, T2 e T4, custo total = 72.
Todos os agentes respeitam suas capacidades (não há excesso).
O custo total dessa nova solução é 230, uma melhora em relação à solução inicial (que era 258).Além disso, diferente da solução anterior (252), aqui não há excesso de capacidade, ou seja, é uma solução viável e mais barata.



PDG – Busca Tabu

	Tarefa							
Agente	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	44	45	•	-4	22	20	3	•
A_2	49	36	16	41	•	•	🚫	-3
A_3	•	🚫	23	•	27	12	22	13



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_8 T_4	0	1	104	2	106	-21	-13	•	•	-7	-15	-22	•
A_2	T_5 T_6 T_7	5	0	88	0	88	-15	-3	-18	-20	•	•	•	-9
A_3	T_1 T_2	7	0	32	0	32	•	•	-11	5	-4	-3	-10	-12
		Custo Total:		**224**	**0**	**226**								



PDG – Busca Tabu

	Tarefa							
Agente	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	48	49	•	🚫	26	24	7	•
A_2	49	36	14	45	•	•	🚫	-5
A_3	•	🚫	17	4	23	8	18	7



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_4	23	0	67	0	67	2	10	•	•	16	8	1	-1
A_2	T_5 T_6 T_7 T_8	0	9	104	18	122	-20	-8	-23	-25	•	•	•	•
A_3	T_1 T_2	7	0	32	0	32	•	•	-11	5	-4	-3	-10	-12
	Custo Total:			**203**	**18**	**221**								

PDG – Busca Tabu

Critério de Aspiração

	Tarefa							
Agente	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	6	23	•	🚫	0	-16	-55	5
A_2	59	46	24	55	•	•	🚫	🚫
A_3	•	•	17	4	11	-2	0	12

O movimento de atribuir a tarefa T7 ao agente A1 teria uma melhoria de −55 no custo, mas está tabu.

O Critério de Aspiração permite ignorar a restrição tabu quando um movimento tabu leva a uma solução melhor que a melhor encontrada até o momento. Ou seja, mesmo que a regra normal proibisse o movimento, ele pode ser aceito porque garante um grande ganho global.



PDG – Busca Tabu

		Capacidade		Custo			Cap. disp. após movimento							
Agente	Tarefas	Disp.	Excesso	Alocação	Excesso	Total	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
A_1	T_3 T_4 T_7	1	0	79	0	79	-20	-12	•	•	-6	-14	•	-23
A_2	T_5 T_6 T_8	0	0	55	0	55	-20	-8	-23	-25	•	•	-9	•
A_3	T_1 T_2	7	0	32	0	32	•	•	-11	5	-4	-3	-10	-12
		Custo Total:		**166**	**0**	**166**								

O custo total agora é 166, bem menor que a solução inicial de 258.
Não há excesso de recursos em nenhum agente (todos respeitam os limites).
a Busca Tabu, ao longo das iterações, conseguiu redistribuir melhor as tarefas.
A solução é viável (sem exceder capacidade) e muito mais barata (redução de 258 → 166, ou seja, 92 unidades a menos).
Isso comprova o poder da Busca Tabu em escapar de soluções locais ruins e encontrar alocações globais melhores.



PDG – Busca Tabu

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	23	38	33	34	26	19	12	37
A2	36	45	13	41	14	25	49	16
A3	17	15	30	40	29	27	47	22

Agente	Tarefa							
	T1	T2	T3	T4	T5	T6	T7	T8
A1	21	13	4	3	7	15	22	24
A2	20	8	23	25	6	5	9	14
A3	16	12	18	2	11	10	17	19



Machine Learning

Pode ser usado para a calibração de hiperparâmetros em modelos de Aprendizagem de Máquina

Por exemplo, o classificador KNN (K Nearest Neighbors) emprega dois hiperparâmetros:

K → número de vizinhos

Weights → ponderação dos vizinhos



Bibliografia

RAPIDMINER. Multi-Objective Feature Selection Part 1: The Basics. Disponível em https://community.rapidminer.com/discussion/45775/multi-objective-feature-selection-part-1-the-basics . Acesso em 18/10/24.

GLOVER, F.; LAGUNA, M. (1997). Tabu Search Background. In: Tabu Search. Springer, Boston, MA. https://doi.org/10.1007/978-1-4615-6089-0_1